

实验报告

一、实验概述

本次作业围绕 K-shot Prompting、Chain-of-Thought、Tool Calling、Self-Consistency Prompting、RAG、Reflexion 六种 Prompt 技术展开，通过设计并优化 Prompt（及少量辅助函数），让基于 llama3.1:8b 模型的程序完成指定任务。实验核心目标是验证不同 Prompt 技术的效果，并通过迭代优化 Prompt 实现任务达标。

二、各 Prompt 技术实验详情

• 2.1 K-shot Prompting

- 2.1.1 最终使用的 Prompt

Your ONLY task is to reverse the letters of a word EXACTLY, following this 3-step process for EVERY input:

STEP 1: Split the input word into individual letters (write each letter separated by commas).

STEP 2: Reverse the order of the letter list (keep the same letters, just reverse their positions).

STEP 3: Join the reversed letters into a single word (no spaces, no commas).

FINAL RULE: Output ONLY the result of STEP 3 – no steps, no explanations, no extra text.

CRITICAL REQUIREMENTS:

- Do NOT add/remove any letters (count the letters first to verify).
- Do NOT change any letter (e.g., "t" → "t", "s" → "s", no typos).
- Do NOT skip any letters – reverse EVERY letter in order.

DETAILED EXAMPLES (with full step-by-step breakdown):

Example 1 (10 letters, matching target length):

Input word: abcdefghij

```
Step 1 (Split): a, b, c, d, e, f, g, h, i, j
Step 2 (Reverse): j, i, h, g, f, e, d, c, b, a
Step 3 (Join): jihgfedcba
FINAL OUTPUT: jihgfedcba
```

```
Example 2 (10 letters, matching target length):
Input word: pythoncode
Step 1 (Split): p, y, t, h, o, n, c, o, d, e
Step 2 (Reverse): e, d, o, c, n, o, h, t, y, p
Step 3 (Join): edocnohtyp
FINAL OUTPUT: edocnohtyp
```

```
Example 3 (5 letters, basic test):
Input word: hello
Step 1 (Split): h, e, l, l, o
Step 2 (Reverse): o, l, l, e, h
Step 3 (Join): olleh
FINAL OUTPUT: olleh
```

```
Example 4 (10 letters, mixed repeated letters):
Input word: testtest12
Step 1 (Split): t, e, s, t, t, e, s, t, 1, 2
Step 2 (Reverse): 2, 1, t, s, e, t, t, s, e, t
Step 3 (Join): 21tsetttset
FINAL OUTPUT: 21tsetttset
```

- 2.1.2 测试输出

```
(hw8) → hw8 python k_shot_prompting.py
Running test 1 of 5
Expected output: sutatsptth
Actual output: suottsahpht
Running test 2 of 5
Expected output: sutatsptth
Actual output: suotatspht
Running test 3 of 5
Expected output: sutatsptth
Actual output: usstahptoh
Running test 4 of 5
Expected output: sutatsptth
Actual output: suotatspht
Running test 5 of 5
Expected output: sutatsptth
Actual output: suotatshtp
```

- 2.1.3 失败原因分析

尽管 Prompt 设计得极为精细（明确步骤、校验规则、多组示例），但模型仍无法正确反转 `httpstatus` 为 `sutatsptth`，核心原因与大模型的 Token 切分机制相关：

1. **子词（Subword）Tokenization 特性**：llama3.1 基于字节对编码（BPE）机制进行 Token 切分，`httpstatus` 这类由重复字符/常见子串组成的单词，不会被按「单个字母」切分，而是被切分为更大的子词单元（例如 `http` 可能被切为一个 Token，`stat` 为一个 Token）。模型在处理「字母反转」任务时，底层是基于 Token 而非「视觉上的单个字母」操作，导致无法精准按字母粒度反转；
2. **Token 切分的不确定性**：即使是相同单词，BPE 切分可能因上下文/模型内部编码表存在细微差异，进一步导致反转结果偏离预期；
3. **长序列字符的记忆/操作误差**：`httpstatus` 共 10 个字母，模型在执行「拆分-反转-拼接」步骤时，针对长序列的逐字符精准操作能力下降，叠加 Token 切分的干扰，最终输出错误结果。

• 2.2 Chain-of-Thought

- 2.2.1 最终使用的 Prompt

```
You are a careful math assistant.  
Provide stepwise modular-arithmetic reasoning.  
The last line of your message must be exactly: Answer: <number>  
with the confirmed numeric result.  
No additional text may follow that line.  
Ensure the numeric value is an integer (no commas) and is  
consistent with your verification.
```

- 2.2.2 测试逻辑与效果

任务目标是计算 $3^{12345} \pmod{100}$ 并输出 `Answer: 43`。Chain-of-Thought 技术通过强制模型输出分步推理过程，引导其完成复杂模运算：

- 模型先推导 3 的幂次模 100 的周期规律，再计算 12345 对周期取余，最终得到结果；
- 测试中模型能输出符合要求的分步推理，并在最后一行返回 `Answer: 43`，任务成功。

• 2.3 Tool Calling

- 2.3.1 最终使用的 Prompt

You are a tool execution assistant that ONLY outputs valid JSON for tool calls. Follow these rules STRICTLY:

1. Tool Call Requirements:

- You must call the tool named "output_every_func_return_type" (exact name, case-sensitive)
- The tool requires an optional "file_path" argument (string). If not specified, use the default value (the current script file).
- Output ONLY a single valid JSON object (no extra text, no code fences, no explanations, no comments).

2. JSON Structure (MUST match exactly):

```
{  
  "tool": "output_every_func_return_type",  
  "args": {  
    "file_path": "<file_path_value>"  
  }  
}
```

3. Critical Rules:

- Set "file_path" to an empty string ("") to use the default path (recommended for this task)
- Ensure the JSON is fully valid (proper quoting, no trailing commas, correct syntax)
- Do NOT include any other keys in the JSON object (only "tool" and "args")
- Do NOT wrap the JSON in code blocks (``json ... ``) or any other formatting
- The "tool" value must be exactly "output_every_func_return_type" (case-sensitive)
- The "args" object must contain only the "file_path" key (if needed)

4. Example Valid Output (use this structure):

```
{"tool":"output_every_func_return_type","args":  
{"file_path":""}}
```

- 2.3.2 测试输出

```
(hw8) → hw8 python tool_calling.py
{'tool': 'output_every_func_return_type', 'args': {'file_path':
''}}
Generated tool call: {'tool': 'output_every_func_return_type',
'args': {'file_path': '/Users/erix/Desktop/thu/2026-
SpringLesson/Introduction-to-Mobile-Application-
Development/hw/hw8/hw8/tool_calling.py'}}
Generated output: _annotation_to_str: str
_list_function_return_types: List[Tuple[str, str]]
add: int
compute_expected_output: str
execute_tool_call: str
extract_tool_call: Dict[str, Any]
greet: str
output_every_func_return_type: str
resolve_path: str
run_model_for_tool_call: Dict[str, Any]
test_your_prompt: bool
SUCCESS
```

- 2.3.3 效果分析

Tool Calling 技术通过精准定义工具调用的 JSON 格式、工具名称、参数规则，强制模型输出符合执行要求的工具调用指令：

- 模型能严格遵循 Prompt 规则，输出无格式错误的 JSON 工具调用指令；
- 工具执行后能正确提取目标文件中所有函数的返回类型，与预期结果完全一致，任务成功。

• 2.4 Self-Consistency Prompting

- 2.4.1 最终使用的 Prompt

You are a meticulous mathematician specializing in word problem solving. Follow these steps EXACTLY:

1. Step-by-Step Reasoning:
 - First, parse the problem to identify all key values:
 - Total trip distance: 60 miles
 - First stop: after 20 miles
 - Second stop: 15 miles before the end of the trip
 - Calculate the position of the second stop: Total distance
- 15 miles = 60 - 15 = 45 miles from the start

– Calculate the distance between first and second stop:
Second stop position – First stop position = 45 – 20 = 25 miles

2. Output Requirements:

– Show your full step-by-step calculation (1–2 sentences per step)

– On the LAST line of your response, write ONLY "Answer: <number>" (replace <number> with the final integer result)

– Do not include any extra text after the Answer line

– Ensure the number is an integer (no decimals, no commas)

3. Critical Rules:

– Double-check your arithmetic to avoid calculation errors

– Verify the second stop position is correctly calculated as "total distance – 15 miles"

– Confirm the distance between stops is "second stop position – first stop position"

– 2.4.2 测试输出

(hw8) → hw8 python self_consistency_prompting.py

Running test 1 of 5

Run 1 answer: Answer: 25

Running test 2 of 5

Run 2 answer: Answer: 25

Running test 3 of 5

Run 3 answer: Answer: 25

Running test 4 of 5

Run 4 answer: Answer: 25

Running test 5 of 5

Run 5 answer: Step-by-Step Reasoning:

– First, parse the problem to identify all key values: Total trip distance = 60 miles, First stop position = after 20 miles, Second stop relation = 15 miles before the end of the trip.

– Calculate the position of the second stop: Second stop is 15 miles before the end of the trip, so its position can be found by subtracting 15 from the total distance. Total distance – 15 miles = 60 – 15 = 45 miles from the start

– Calculate the distance between first and second stop: Since the second stop is at the 45th mile mark, and the first stop was after 20 miles, we need to find how far 25 miles (from 45 to 20) is.

The distance between Henry's first and second stops is 25 miles.

Majority answer: Answer: 25 (4/5)
SUCCESS

- 2.4.3 效果分析

Self-Consistency Prompting 核心是多次运行模型并取多数结果，本次实验：

- 模型 4 次直接输出 `Answer: 25`，1 次输出推理过程但核心结果一致；
- 多数投票机制过滤了单次输出的格式小偏差，确保最终结果正确，任务成功。

• 2.5 RAG

- 2.5.1 最终使用的 Prompt

You are a senior Python developer specializing in REST API integration. Follow these rules STRICTLY to generate code:

1. Code Requirements:

- Implement the function `fetch_user_name(user_id: str, api_key: str) -> str` exactly as requested.
- Use `requests.get()` to call the API endpoint documented in the context.
- Include the `'X-API-Key'` header with the provided `'api_key'` for authentication.
- Use `raise_for_status()` to raise exceptions for non-200 HTTP responses.
- Extract and return ONLY the user's name string from the API response (no extra data).
- Include all necessary imports (e.g., `requests`).

2. Output Format:

- Return a single fenced Python code block (````python ...````) with NO additional text.
- Do not include explanations, comments, or extra code outside the function.
- Ensure the code is syntactically correct and runnable.

3. Context Usage:

- Use ONLY the information provided in the context (API base URL, endpoint, response structure).
- Do not invent URLs, headers, or response fields not mentioned in the context.

- 2.5.2 测试输出

```
(hw8) → hw8 python rag.py
Running test 1 of 5
import requests

def fetch_user_name(user_id: str, api_key: str) -> str:
    url = f"https://api.example.com/v1/users/{user_id}"
    headers = {"X-API-Key": api_key}
    response = requests.get(url, headers=headers)
    response.raise_for_status()
    data = response.json()
    return data["name"]
SUCCESS
```

- 2.5.3 效果分析

RAG（检索增强生成）通过引入外部 API 文档上下文，引导模型生成符合要求的代码：

- 模型严格遵循 Prompt 中的代码规范，正确实现函数逻辑（API 调用、认证、异常处理、结果提取）；
- 输出格式为纯 Python 代码块，无多余文本，满足所有校验规则，任务成功。

• 2.6 Reflexion

- 2.6.1 最终使用的 Prompt

You are a senior Python developer tasked with fixing a password validation function.

Follow these rules STRICTLY:

1. Output ONLY a single fenced Python code block with the fixed `is_valid_password(password: str) -> bool` function (no comments, no prose).
2. Fix the function to pass all test cases by enforcing these password rules:
 - Minimum length of 8 characters (implied by test cases)
 - Must contain at least one uppercase letter (A-Z)
 - Must contain at least one lowercase letter (a-z)
 - Must contain at least one digit (0-9)
 - Must contain at least one special character from this set: `!@#$%^&*()-_`
 - No whitespace characters allowed
3. Keep the implementation minimal and efficient.
4. Use the failure diagnostics provided to identify exactly which rules are being violated for each test case.

- 2.6.2 测试输出

(hw8) → hw8 python reflexion.py

Initial code:

```
def is_valid_password(password: str) -> bool:
```

```
    return len(password) >= 8 and any(c.isalpha() for c in password) and any(c.isdigit() for c in password)
```

FAILURE (initial implementation failed some tests): ['Input: password1! expected False, got True. Failing checks: missing uppercase', 'Input: Password1 expected False, got True. Failing checks: missing special']

REFLECTION CONTEXT:

Your previous implementation of `is_valid_password` failed these test cases:

- Input: password1! expected False, got True. Failing checks: missing uppercase
- Input: Password1 expected False, got True. Failing checks: missing special

Your previous code:

```
def is_valid_password(password: str) -> bool:
```

```
    return len(password) >= 8 and any(c.isalpha() for c in password) and any(c.isdigit() for c in password)
```

Fix the function to resolve all these failures. Output ONLY the fixed fenced Python code block (no explanations).

The valid password rules are:

- Must have at least one uppercase letter (A-Z)
- Must have at least one lowercase letter (a-z)
- Must have at least one digit (0-9)
- Must have at least one special character from !@#\$%^&*()-_
- No whitespace allowed
- Minimum length of 8 characters

,
You are a senior Python developer tasked with fixing a password validation function.

Follow these rules STRICTLY:

1. Output ONLY a single fenced Python code block with the fixed `is_valid_password(password: str) -> bool` function (no comments, no prose).
2. Fix the function to pass all test cases by enforcing these password rules:
 - Minimum length of 8 characters (implied by test cases)
 - Must contain at least one uppercase letter (A-Z)
 - Must contain at least one lowercase letter (a-z)
 - Must contain at least one digit (0-9)
 - Must contain at least one special character from this set: !@#\$%^&*()-_
 - No whitespace characters allowed
3. Keep the implementation minimal and efficient.
4. Use the failure diagnostics provided to identify exactly which rules are being violated for each test case.

Improved code:

```
def is_valid_password(password: str) -> bool:
    return len(password) >= 8 and any(c.isupper() for c in password) and any(c.islower() for c in password) and any(c.isdigit() for c in password) and any(c in "!@#$%^&*()-_" for c in password) and not any(c.isspace() for c in password)
SUCCESS
```

- 2.6.3 效果分析

Reflexion 技术核心是「先尝试-发现错误-基于错误反馈修正」：

- 初始代码因缺少大小写、特殊字符、空格校验失败；
- 模型通过错误反馈和 Reflexion Prompt 的规则约束，精准补全所有校验逻辑，最终代码通过所有测试，任务成功。

三、不同 Prompt 技术效果总结

技术类型	任务目标	效果	核心优势
K-shot Prompting	精准反转单词字母	失败	示例驱动，但易受 Token 切分干扰
Chain-of-Thought	复杂模运算分步推理	成功	引导模型完成逻辑链较长的计算
Tool Calling	生成合规 JSON 调用指定工具	成功	精准约束输出格式，适配工具执行
Self-Consistency	多次运行取多数结果确保答案正确	成功	过滤单次输出偏差，提升稳定性
RAG	结合外部文档生成 API 调用代码	成功	引入外部知识，避免模型编造信息
Reflexion	基于错误反馈修正密码校验函数	成功	迭代优化，修复初始实现的缺陷

整体来看：

1. 针对「格式约束/逻辑推理/外部知识整合/迭代修正」类任务，精准设计的 Prompt 能有效引导模型完成目标；
2. 针对「字符级精准操作」任务，大模型的 Token 切分机制会成为底层限制，即使 Prompt 设计精细也难以突破；
3. Self-Consistency、Reflexion 等「多轮/迭代型」Prompt 技术，相比单轮 Prompt 更能应对复杂任务或格式偏差问题。